

```
from google.colab import drive
drive.mount('/content/drive')

import os
DRIVE_DIR = '/content/drive/MyDrive/mini-clip-dlai'
os.makedirs(DRIVE_DIR, exist_ok=True)
print('Drive mounted. Project folder:', DRIVE_DIR)
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)
Drive mounted. Project folder: /content/drive/MyDrive/mini-clip-dlai

✓ Clone repo & install dependencies

```
!git clone https://github.com/prajwl7676/mini-clip-DLAI.git

%cd /content/mini-clip-DLAI

!pip install -q -r requirements.txt

print('Setup complete.')
```

fatal: destination path 'mini-clip-DLAI' already exists and is not an empty directory.
/content/mini-clip-DLAI
Setup complete.

✓ Imports & device setup

```
import torch
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
from datasets import load_dataset
from transformers import DistilBertTokenizer
from torchvision import transforms
from torch.utils.data import Dataset, DataLoader

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'PyTorch version : {torch.__version__}')
print(f'Device          : {device}')
if device.type == 'cuda':
    print(f'GPU          : {torch.cuda.get_device_name(0)}')
```

PyTorch version : 2.5.1+cu124
Device : cuda
GPU : Tesla T4

✓ Load Flickr30k from HuggingFace (10k samples)

```
##Load Flickr30k from HuggingFace (10k samples)

full_dataset = load_dataset('nlphuji/flickr30k', split='test')

print(f'Total samples in dataset : {len(full_dataset)}')
print(f'Column names              : {full_dataset.column_names}')
print(f'Features                    : {full_dataset.features}')
```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
Total samples in dataset : 31014

```
Column names      : ['image', 'caption', 'sentids', 'split', 'img_id', 'filename']
Features         : {'image': Image(mode=None, decode=True, id=None), 'caption': [Value(dtype='string', id=None)], 'sen
```

▼ Inspect one example

```
example = full_dataset[0]

print('Keys in one example:', list(example.keys()))

print('Image type :', type(example['image']))
print('Image size :', example['image'].size)
print('Image mode :', example['image'].mode)
print()
print('Number of captions:', len(example['caption']))
print('Captions:')
for i, cap in enumerate(example['caption']):
    print(f' [{i}] {cap}')
```

Keys in one example: ['image', 'caption', 'sentids', 'split', 'img_id', 'filename']
Image type : <class 'PIL.JpegImagePlugin.JpegImageFile'>
Image size : (333, 500)
Image mode : RGB

Number of captions: 5
Captions:
[0] Two young guys with shaggy hair look at their hands while hanging out in the yard.
[1] Two young, White males are outside near many bushes.
[2] Two men in green shirts are standing in a yard.
[3] A man in a blue shirt standing in a garden.
[4] Two friends enjoy time spent together.

▼ Visualise 6 examples

```
import os

os.makedirs('assets', exist_ok=True)
fig, axes = plt.subplots(2, 3, figsize=(14, 9))
axes = axes.flatten()

for i, ax in enumerate(axes):
    item = full_dataset[i]
    image = item['image']
    caption = item['caption'][0]    # caption 0 out of 5

    ax.imshow(image)
    ax.set_title(
        caption if len(caption) <= 60 else caption[:57] + '...',
        fontsize=9, wrap=True
    )
    ax.axis('off')

plt.suptitle('Flickr30k - 6 example image-caption pairs', fontsize=12)
plt.tight_layout()
plt.savefig('assets/flickr_examples.png', dpi=120, bbox_inches='tight')
plt.show()
print('Saved to assets/flickr_examples.png')
```

Flickr30k — 6 example image-caption pairs

Two young guys with shaggy hair look at their hands while...



Several men in hard hats are operating a giant pulley sys...



A child in a pink dress is climbing up a set of stairs in...



Someone in a blue shirt and hat is standing on stair and ...



Two men, one in a gray shirt, one in a black shirt, stand...



Two people in the photo are playing the guitar and the ot...



Saved to assets/flickr_examples.png

- Split into train / val / test-- 80 / 10 / 10.

```
TOTAL = 10_000
N_TRAIN = 8_000
N_VAL = 1_000
# test = remaining 1,000

dataset_10k = full_dataset.select(range(TOTAL))

train_hf = dataset_10k.select(range(0, N_TRAIN))
val_hf = dataset_10k.select(range(N_TRAIN, N_TRAIN + N_VAL))
test_hf = dataset_10k.select(range(N_TRAIN + N_VAL, TOTAL))

print(f'Train : {len(train_hf):,} samples')
print(f'Val : {len(val_hf):,} samples')
print(f'Test : {len(test_hf):,} samples')
```

```
Train : 8,000 samples
Val : 1,000 samples
Test : 1,000 samples
```

- Load the tokenizer

The tokenizer converts a caption string into a sequence of integer IDs that the DistilBERT text encoder can process.

```
tokenizer = DistilBertTokenizer.from_pretrained('distilbert-base-uncased')

# --- see what tokenisation looks like ---
sample_caption = train_hf[0]['caption'][0]
sample_image = train_hf[0]['image']
plt.imshow(sample_image)
plt.axis('off')
plt.show()
```

```
print('Image size :', sample_image.size)
print('Caption :', sample_caption)
print()
```



Image size : (333, 500)

Caption : Two young guys with shaggy hair look at their hands while hanging out in the yard.

```
encoded = tokenizer(
    sample_caption,
    padding='max_length',
    max_length=77,
    truncation=True,
    return_tensors='pt',
)
print('input_ids shape      :', encoded['input_ids'].shape)      # (1, 77)
print('attention_mask shape :', encoded['attention_mask'].shape) # (1, 77)
print()
print('input_ids (first 15 tokens):', encoded['input_ids'][0, :15].tolist())
print('Decoded back          :', tokenizer.decode(encoded['input_ids'][0]))
```

```
input_ids shape      : torch.Size([1, 77])
attention_mask shape : torch.Size([1, 77])
```

```
input_ids (first 15 tokens): [101, 2048, 2402, 4364, 2007, 25741, 2606, 2298, 2012, 2037, 2398, 2096, 5689, 2041, 1999]
Decoded back          : [CLS] two young guys with shaggy hair look at their hands while hanging out in the yard. [SEP] [PAD] [P.
```

✓ Define image transforms

ResNet-50 expects **224 × 224** images, normalised with ImageNet statistics.

Using slightly different transforms for training (with augmentation) vs eval.

```
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD  = [0.229, 0.224, 0.225]
```

```

train_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.RandomCrop(224),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
])

eval_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
])

```

```

# Test on one image
sample_image = train_hf[0]['image'].convert('RGB')
tensor = train_transform(sample_image)

print('Original image size :', sample_image.size)  # (W, H)
print('After transform      :', tensor.shape)      # (3, 224, 224)
print('Pixel value range   : [{:.2f}, {:.2f}]'.format(tensor.min().item(), tensor.max().item()))

Original image size : (333, 500)
After transform      : torch.Size([3, 224, 224])
Pixel value range    : [-2.10, 2.64]

```

✓ Write the FlickrDataset class

src/dataset.py.

```

class FlickrDataset(Dataset):

    def __init__(self, hf_dataset, tokenizer, transform, max_length=77, caption_idx=0):
        self.data      = hf_dataset
        self.tokenizer  = tokenizer
        self.transform  = transform
        self.max_length = max_length
        self.caption_idx = caption_idx

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        item = self.data[idx]

        # Image
        image = item['image']
        if image.mode != 'RGB':
            image = image.convert('RGB')
        image = self.transform(image)          # (3, 224, 224)

        # Caption
        captions = item['caption']
        caption = captions[self.caption_idx] if isinstance(captions, list) else captions

        # Tokenise
        encoded = self.tokenizer(
            caption,
            padding='max_length',
            max_length=self.max_length,
            truncation=True,
            return_tensors='pt',
        )
        tokens = {
            'input_ids': encoded['input_ids'].squeeze(0),
            'attention_mask': encoded['attention_mask'].squeeze(0),
        }
        return image, tokens

```

```
print('FlickrDataset class defined.')
```

```
FlickrDataset class defined.
```

✓ Test the Dataset

Before building a DataLoader, verify that a single `__getitem__` call returns the correct shapes.

```
train_dataset = FlickrDataset(train_hf, tokenizer, train_transform)
val_dataset   = FlickrDataset(val_hf,   tokenizer, eval_transform)
test_dataset  = FlickrDataset(test_hf,  tokenizer, eval_transform)
```

```
print(f'Train dataset size : {len(train_dataset):,}')
print(f'Val   dataset size : {len(val_dataset):,}')
print(f'Test  dataset size : {len(test_dataset):,}')
```

```
Train dataset size : 8,000
Val   dataset size : 1,000
Test  dataset size : 1,000
```

```
# Fetch one item and check shapes
image, tokens = train_dataset[0]
```

```
print('image shape      : ', image.shape)           # (3, 224, 224)
print('input_ids shape  : ', tokens['input_ids'].shape) # (77,)
print('attention_mask shape: ', tokens['attention_mask'].shape) # (77,)
print('image dtype      : ', image.dtype)           # torch.float32
print('input_ids dtype  : ', tokens['input_ids'].dtype) # torch.int64
```

```
# Sanity check
assert image.shape == (3, 224, 224), 'Wrong image shape!'
assert tokens['input_ids'].shape == (77,), 'Wrong token shape!'
print()
print('All shape checks passed.')
```

```
image shape      : torch.Size([3, 224, 224])
input_ids shape  : torch.Size([77])
attention_mask shape: torch.Size([77])
image dtype      : torch.float32
input_ids dtype  : torch.int64
```

```
All shape checks passed.
```

✓ Build DataLoaders and test a batch

A `DataLoader` wraps a `Dataset` and handles batching, shuffling, and parallel loading. `batch_size=64`

```
BATCH_SIZE = 64
NUM_WORKERS = 2

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=NUM_WORKERS,
    pin_memory=True,    # faster CPU->GPU memory transfer
    drop_last=True,
)
val_loader = DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS,
    pin_memory=True,
)
```

```
test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS,
    pin_memory=True,
)

print(f'Train batches : {len(train_loader)}')
print(f'Val   batches : {len(val_loader)}')
print(f'Test   batches : {len(test_loader)}')
```

```
Train batches : 125
Val   batches : 16
Test   batches : 16
```

```
# Fetch one batch and verify shapes
images_batch, tokens_batch = next(iter(train_loader))

print('--- One batch from train_loader ---')
print('images shape      : ', images_batch.shape)          # (64, 3, 224, 224)
print('input_ids shape   : ', tokens_batch['input_ids'].shape) # (64, 77)
print('attention_mask shape: ', tokens_batch['attention_mask'].shape) # (64, 77)
```

```
--- One batch from train_loader ---
images shape      : torch.Size([64, 3, 224, 224])
input_ids shape   : torch.Size([64, 77])
attention_mask shape: torch.Size([64, 77])
```

✓ Visualise a batch (un-normalise for display)

After normalisation, pixel values are no longer in $[0, 1]$ — reverse the transform to display images correctly.

```
def unnormalise(tensor):
    """Reverse ImageNet normalisation for display. Input: (3, H, W) tensor."""
    mean = torch.tensor(IMAGENET_MEAN).view(3, 1, 1)
    std = torch.tensor(IMAGENET_STD).view(3, 1, 1)
    return (tensor * std + mean).clamp(0, 1)

fig, axes = plt.subplots(2, 4, figsize=(14, 7))
axes = axes.flatten()

for i, ax in enumerate(axes):
    img_tensor = unnormalise(images_batch[i])          # (3, 224, 224)
    img_np      = img_tensor.permute(1, 2, 0).numpy() # (224, 224, 3)

    # Decode the caption tokens back to text
    ids      = tokens_batch['input_ids'][i]
    caption = tokenizer.decode(ids, skip_special_tokens=True)
    caption = caption[:55] + '...' if len(caption) > 55 else caption

    ax.imshow(img_np)
    ax.set_title(caption, fontsize=8)
    ax.axis('off')

plt.suptitle('Batch sample – images with decoded captions', fontsize=11)
plt.tight_layout()
plt.savefig('assets/batch_sample.png', dpi=120, bbox_inches='tight')
plt.show()
print('Saved to assets/batch_sample.png')
```


Batch sample — images with decoded captions

a man kneels on the ground doing some sort of project o...



people at a concert wave their hands in the air as they...



a black male leaping over a hurdle on a track during a ...



a blond woman in a white tank top and sunglasses is tal...



a woman is laughing as two small boys play around her a...



two children are riding in a red play vehicle at a real...



a man with long hair pulled into a ponytail and thick f...



a group of young asian men leaning on a balcony, two in...



Saved to assets/batch_sample.png

✓ Caption length analysis

Understanding caption lengths helps choose `max_length=77` wisely.

```
# Sample 500 captions and measure their token lengths
lengths = []
for i in range(500):
    cap = train_hf[i]['caption'][0]
    toks = tokenizer(cap, truncation=False)['input_ids']
    lengths.append(len(toks))

lengths = np.array(lengths)
```

```
fig, ax = plt.subplots(figsize=(9, 4))
ax.hist(lengths, bins=30, color='steelblue', edgecolor='white', linewidth=0.5)
ax.axvline(77, color='crimson', linestyle='--', linewidth=1.5, label='max_length=77')
ax.set_xlabel('Caption length (tokens)')
ax.set_ylabel('Count')
ax.set_title('Distribution of caption token lengths (500 samples)')
ax.legend()
plt.tight_layout()
plt.savefig('assets/caption_lengths.png', dpi=120, bbox_inches='tight')
plt.show()

print(f'Mean   : {lengths.mean():.1f} tokens')
print(f'Median : {np.median(lengths):.1f} tokens')
print(f'Max    : {lengths.max():.1f} tokens')
print(f'% > 77 : {(lengths > 77).mean() * 100:.1f}%')
```


